IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

RÉPUBLIQUE
FRANÇAISE
Liberté
Égalité
Fraternité

Quantum Generative Adversarial Networks Hybrid Architectures and Convergence Analysis

Leveraging Variational Quantum Circuits for High-Dimensional Data Synthesis

Antony Manuel

IMT Nord Europe
Department of Computer Science
Master of Data Science

Lille, April 2026

Abstract

This report describes the work carried out during the internship of the first year of my master degree at Numalis. For several years, Deep Learning has enabled major breakthrough in machine learning. Nevertheless, adversarial examples remain a major issue for machine learning models in many applications. This work aims to produce an adversarial examples detection method based on explainability, such as CRAFT method developed by Fel et al. [1] which offer the opportunity to observe the elements models qualified as relevant in his prediction. Finally the results obtained illustrate balanced effectiveness, depending on the attack used to generate adversarial inputs. However, it paves the way to new defenses method relying on explainability.

Keywords: Quantum computing, Parameterized Quantum Circuits, Generative Adversarial Networks, Quantum Machine Learning, Quantum Generative Adversarial Networks, Training Stability

Notation

$\mathcal{X}$	Set.
$\mathbb{N}$	Set of integers.
$\mathbb{R}$	Set of real numbers.
$\subset$	Strict inclusion.
$\subseteq$	Inclusion with possible equality.
$\lvert \cdot \rvert$	Absolute value of a real number.
$\lVert \cdot \rVert$	Euclidean norm of a vector.
x	A real scalar belonging to $\mathbb{R}$.
$x \in \mathbb{R}^n$	A vector in $\mathbb{R}^n$.
$X \in \mathbb{R}^{n \times d}$	A matrix in $\mathbb{R}^{n \times d}$.
I_n	Identity matrix of size n.
$X^\top$	Transpose of X.
X^{-1}	Inverse of X.
x	A random scalar variable.
$x \in \mathbb{R}^n$	A random vector in $\mathbb{R}^n$.
$X \in \mathbb{R}^{n \times d}$	A random matrix in $\mathbb{R}^{n \times d}$.
$P(x)$, $P(\mathbf{x})$	Probability distribution of x (resp. joint probability distribution of $\mathbf{x}$).
$x \sim P_x$	Random variable x follows a probability distribution P_x.
$\mathbb{E}(x)$	Expectation of x with respect to $P(x)$.
$\mathrm{Var}(x)$	Variance of x under $P(x)$.
x_i	i-th element of the vector x.
$X_{i,j}$	Element at row i, column j of the matrix X.
$X_{i,:}$	i-th row of the matrix X.
$X_{:,i}$	i-th column of the matrix X.
$f \circ g$	Composition of functions f and g.
dy/dx	Derivative of y with respect to x.
$\partial y/\partial x$	Partial derivative of y with respect to x.
$\partial f/\partial x$	Jacobian matrix $J \in \mathbb{R}^{n \times p}$ of $f : \mathbb{R}^n \to \mathbb{R}^p$.

Sometimes we use a function f whose argument is a scalar but apply it to a vector $f(x)$ or a matrix $f(X)$. This denotes the element-wise application of f to the array.

$$* \ * \ *$$

Contents

Contents **3**

1 Introduction **5**

2 Background **7**
1 Generative Modeling . . . 7
1.1 Generative Adversarial Networks . . . 7
1.2 Wasserstein Generative Adversarial Networks . . . 8
2 Quantum computing . . . 9
2.1 Quantum Bits . . . 9
2.2 Quantum gates and circuits . . . 10
2.2.1 Single Qubits Gates . . . 10
2.2.2 Multi Qubit Gates . . . 11
2.3 Variational Quantum Algorithms . . . 11

3 Qgan framework **12**
1 Architecture overview . . . 12
1.1 Generator logic . . . 12
1.2 Critic structure . . . 12
2 Training Strategy . . . 13
2.1 Training objective . . . 13
2.2 Training algorithm . . . 13
3 Experimental Setup . . . 13
3.1 Dataset and libraries . . . 13
3.2 Hyperparameters . . . 13

4 Results **14**
1 Results Analysis . . . 14
2 Discussion . . . 14

5 Conclusion **15**

Bibliography **16**

Annexes **17**

A Appendix **18**
1 Exemple 18

B Annexe **20**
1 Cost functions 20
2 Measurement 20
3 Optimizers 20
4 Gradient descent 20

Introduction

Over the past decade, generative models have emerged as one of the most dynamic areas of development within deep learning, continually finding novel applications across diverse domains, including image synthesis, musical composition, and drug discovery. Spanning from Variational Autoencoders (VAEs) to autoregressive models, the field of generative modeling has yielded a plethora of architectures and techniques. Each paradigm possesses distinct strengths and limitations, collectively pushing the boundaries of machine-driven creation. Among these varied approaches, Generative Adversarial Networks (GANs), introduced in 2014 [2], have stood out for their remarkable capacity to generate highly realistic data. GANs have been successfully deployed across a wide spectrum of applications, such as image synthesis [2], text generation, molecular and physical data simulation, and 3D object generation. However, the deployment of classical GANs is constrained by several significant challenges, notably mode collapse, vanishing gradients, extreme sensitivity to hyperparameter selection, and an immense demand for both data and computational resources. Numerous architectural and algorithmic enhancements have been proposed to mitigate these issues [3], for instance Wasserstein GANs [4] employ a distinct distance metric to formulate a loss function that improves training stability, while Karras et al. [5] proposed to incrementally scales the resolution of generated images during the training phase. Despite these advancements, training classical GANs remains inherently complex and computationally expensive, frequently necessitating exhaustive empirical tuning to achieve optimal convergence.

Concurrently, quantum computing has emerged as a transformative computational paradigm, holding the promise of solving certain classes of problems that are intractable for classical computers, such as Shor's algorithm for prime factorization and Grover's algorithm for unstructured database search. Driven by these theoretical advantages, there has been a burgeoning interest in exploring the intersection of quantum computing and machine learning. This synthesis has catalyzed the development of Quantum Machine Learning (QML) algorithms, which aim to harness quantum speedups for complex pattern recognition and optimization tasks. Within the QML landscape, extending quantum methodologies to generative modeling presents a particularly compelling frontier, culminating in the conceptualization of Quantum Generative Adversarial Networks (QGANs) [6, 1]. By replacing traditional neural networks with parameterized quantum circuits (PQCs)

[7] within the generator, the discriminator, or both, QGANs theoretically possess greater representational capacity. This quantum advantage could enable the learning of highly complex, high-dimensional probability distributions more efficiently than classical frameworks, potentially alleviating the severe computational bottlenecks inherent to traditional GAN training.

In this work, we consider the following research papers [8, 9, 10] which propose novel QGAN architectures and training methodologies that aim to enhance the stability and performance of quantum generative models.

Background

1 Generative Modeling

In machine learning, generative modeling is a type of unsupervised learning focused on learning the underlying distribution of a dataset to generate new samples that resemble the original data. Generative models can be broadly categorized into explicit and implicit frameworks. Explicit generative models aim to define or approximate the explicit probability distribution of the data, allowing for likelihood evaluation and direct sampling; examples include Variational Autoencoders (VAEs) [11] and diffusion models [12]. Conversely, implicit generative models, such as Generative Adversarial Networks (GANs) [2], learn to generate realistic samples through adversarial training without explicitly modeling the underlying probability density function. These models span a wide range of domains, including image synthesis, text generation, and data augmentation, and are frequently utilized in financial environments, healthcare, and molecular biology. Because GANs are the primary focus of this research, the following sections provide a detailed overview of their architecture, training process, and subsequent architectural improvements.

1.1 Generative Adversarial Networks

Introduced by Goodfellows et al.[2], Generative Adversarial Networks are models able to learn the distributed probabilities underlying training data and generate new data based on that probability. Moreover GANs are really usefull for understanding distribution probability in unlabeled data, and generate data that are not present in the initial training set.

Basically, a GAN consists of two models, a generator G that outputs synthetic samples $\hat{x}$ given a noise variable input z, bringing potential diversity in the generating process, and is trained to capture the real data distribution in order to generate samples as close as possible to train data. The second model is a discriminator D that estimates the probability of a given sample x being synthetic or coming from the real dataset.

This system inspired from the game theory implies two scenario. The first one emphasizes the goal of the discriminator to classify the input x as real or fake by minimizing

the difference between its output probability $D(x) = P(x|y)$ and the true label y. The second scenario appears when noise inputs z to the generator are randomly sampled from the model's prior over the latent variables, then the discriminator receives input $G(z) = \hat{x}$, a fake sample created by the generator, then the discriminator strives to make $D(\hat{x})$ approach 0, while the generator strives to make the same quantity approach 1.

At the end, if both models have sufficient abilities, then the game reached the Nash equilibrium, in other words, it's corresponding to P_G, the learned distribution matching P_{data} the real then D having a probability $1/2$ of correctly classifying real and fake samples, which means G is able to produce samples indistinguishable from the real data, and D is unable to differentiate between them.

The loss of the model is based on two components, the generator loss and the discriminator loss

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{data}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(\hat{x}))] \tag{2.1}$$

1.2 Wasserstein Generative Adversarial Networks

As previously mentioned, training classical GANs is notoriously difficult, frequently suffering from mode collapse, vanishing gradients, and extreme sensitivity to hyperparameter configurations. To mitigate these pathologies, Arjovsky et al. [4] proposed the Wasserstein GAN (WGAN), which substitutes the traditional Jensen-Shannon divergence with the Wasserstein distance. To formalize this approach while ensuring training stability, Gulrajani et al. [13] introduced the WGAN with Gradient Penalty (WGAN-GP). Under this framework, the Discriminator no longer classifies inputs with a probability $p \in [0, 1]$; instead, it acts as a *critic* D that outputs a scalar score representing the *realness* of the data. Therefore, the objective function of WGAN-GP is formulated as:

$$\min_G \max_{D\in\mathcal{D}} V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[D(x)] - \mathbb{E}_{z\sim p_z}[D(G(z))] - \lambda \mathbb{E}_{\hat{x}\sim p_{\hat{x}}}\left[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2\right] \tag{2.2}$$

where the first two terms compute the original Wasserstein distance, and the third term represents the gradient penalty weighted by the hyperparameter λ. The penalty is evaluated on interpolated samples $\hat{x} = \epsilon x + (1-\epsilon)G(z)$, where $\epsilon \sim \mathcal{U}[0, 1]$. This penalty enforces a 1-Lipschitz constraint on the Critic by penalizing deviations of the gradient norm from 1, effectively preventing the gradients from vanishing or exploding.

Incorporating the WGAN-GP framework is particularly critical when designing a patch-based Quantum GAN (QGAN) for generating structural data like 28×28 MNIST digits. First, the Lipschitz constraint regularizes the critic's landscape, preventing its gradients from becoming too steep, which directly mitigates the severe issue of barren plateaus (narrow gradients) inherent to Parameterized Quantum Circuits (PQCs). Second, since a patch-based architecture generates independent sub-regions of the image, a standard

GAN loss often struggles with spatial coherence, resulting in a disjointed, mosaic-like final output. By providing a continuous real-valued score instead of a binary classification, the WGAN-GP critic offers smoother and more informative gradient feedback. This guides the quantum generator to effectively learn global spatial correlations across boundaries, ensuring seamless integration of the patches.

2 Quantum computing

This section provides a brief overview of the fundamental concepts of quantum computing. It begins with an introduction to quantum information and the representation of quantum states, followed by the description of quantum gates and circuits. Finally, it introduces the concept of Variational Quantum Algorithms (VQAs).

2.1 Quantum Bits

In quantum physics, each quantum state ψ of each quantum system can be represented by a ket $|\psi\rangle$, up to a global phase $e^{i\varphi}$, which is a vector of an Hilbert space $\mathcal{H}$ with its norm equal to 1.

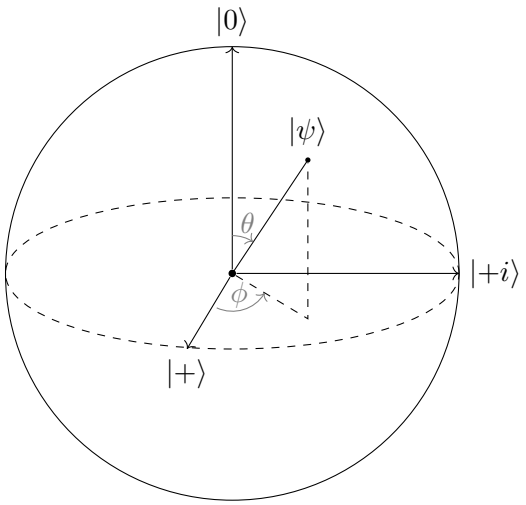


Fig. 2.1: Bloch Sphere

Unlike a classical bit, which is restricted to a binary state $b \in \{0, 1\}$, a qubit exists as a unit vector in a two-dimensional complex Hilbert space $\mathcal{H} \cong \mathbb{C}^2$. Using the computational basis $\mathcal{B} = \{|0\rangle, |1\rangle\}$, a general state $|\psi\rangle$ is expressed as a linear combination:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle \tag{2.3}$$

where $\alpha, \beta \in \mathbb{C}$ are probability amplitudes. According to the Born rule, the probability of measuring the state in $|0\rangle$ is $|\alpha|^2$ and in $|1\rangle$ is $|\beta|^2$, requiring the normalization condition $|\alpha|^2 + |\beta|^2 = 1$. Let's write the amplitudes in polar form, such as $\alpha = ae^{i\varphi}$ and $\beta = be^{i\gamma}$, where a, $b \in \mathbb{R}$. Hence, we can rewrite $|\psi\rangle$ as

$$|\psi\rangle = e^{i\varphi}(a\,|0\rangle + be^{(\gamma-\varphi)}\,|1\rangle) \tag{2.4}$$

since the global phase, $e^{i\varphi}$, does not affect the outcome measurement probabilities, and that 2.4 describe the exact same state :

$$|\psi\rangle = a\,|0\rangle + be^{\phi}\,|1\rangle \tag{2.5}$$

Additionally, the normalization of $|\psi\rangle$ requires that $a^2 + b^2 = 1$ in the way, we can parameterize the state using two real angles θ and ϕ:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle \tag{2.6}$$

where $\theta \in [0, \pi]$ represents the latitude and $\phi \in [0, 2\pi)$ the longitude. As a result, the state of a single qubit can be visualized as a vector in the Bloch sphere 2.1.

2.2 Quantum gates and circuits

In quantum computing, the manipulation of qubit states is executed through quantum circuits, i.e., computational models consisting of a sequence of quantum gates applied to a register of qubits. Mathematically, these gates are represented by unitary operators U (where $U^\dagger U = I$), a property that preserves the norm of the quantum state, thereby ensuring that the transformation is completely reversible and the total probability strictly remains equal to 1.

2.2.1 Single Qubits Gates

As the qubits evolve within the circuit, their state transformation mathematically corresponds to multiplying the state vector by the unitary matrix. Several gates are commonly used, such as the Pauli gates (X, Y, Z) and the Hadamard gate (H):

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Consequently, the transition from an initial state $|\psi_0\rangle$ to a final state $|\psi_f\rangle$ is governed by the equation $|\psi_f\rangle = U\,|\psi_0\rangle$, where U denotes the unitary operator corresponding to the sequence of gates applied within the circuit.

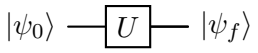


Fig. 2.2: Representation of a gate U applied to a state $|\psi_0\rangle$ resulting in a state $|\psi_f\rangle$.

2.2.2 Multi Qubit Gates

CNOT

2.3 Variational Quantum Algorithms

In addition to these basic gates, parameterized rotation gates $R_x(\theta)$, $R_y(\theta)$, and $R_z(\theta)$ enable arbitrary rotations on the Bloch sphere, as depicted in Fig. 2.3.

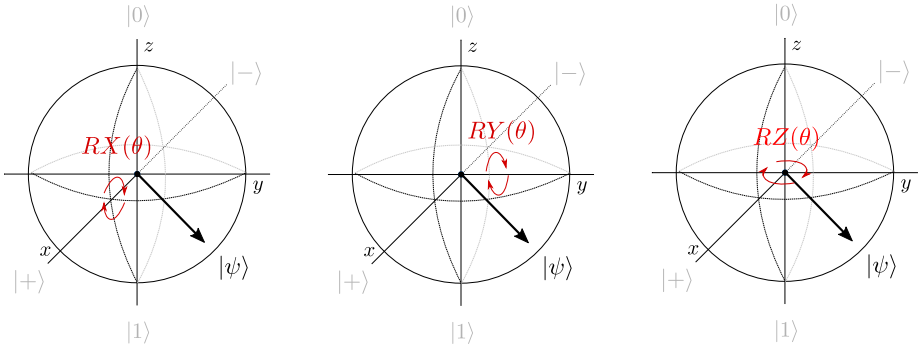


Fig. 2.3: Rotation gates on the Bloch sphere.

Note. Illustration from Pennylane codebook more reference here.

Therefore, a Variational Quantum Circuit (VQC), also known as an ansatz, incorporates these parameterized gates. The underlying parameters are optimized using classical optimization algorithms to minimize a cost function. This hybrid quantum-classical approach, through Variational Quantum Algorithms (VQAs)[14], allows us to leverage the computational advantages of quantum circuits while utilizing classical resources for optimization, which represents a cornerstone approach in quantum machine learning and quantum optimization.

In the context of a QGAN, the generator typically relies on such parameterized quantum circuits. It applies rotation gates to steer the state vector across the Bloch sphere, aiming to align the output with the target data distribution. The ultimate objective is to optimize these parameters so that the resulting quantum measurement statistics become indistinguishable from the real data distribution provided to the discriminator.

Qgan framework

1 Architecture overview

1.1 Generator logic

display and explain generator algorithm

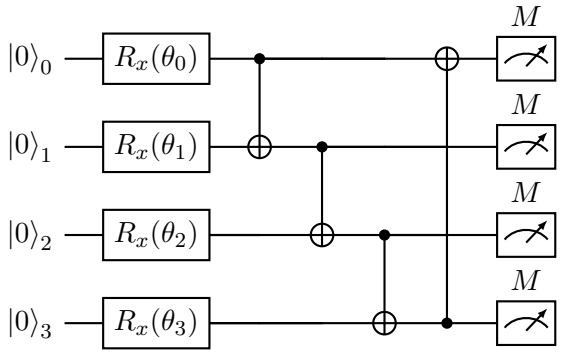


Fig. 3.1: Basic Entangling Ansatz circuit for 4 qubits. Here, θ_i represents the parameter `params[0][i]`, and M represents the observable.

We will build a **Variational Quantum Circuit**. The "noise" (latent vector) is encoded using initial rotations, and the "weights" are the parameters the classical optimizer will update.

Here is a functional template for a 4-qubits generator (producing a 2×2 or 4×4 patch). You can scale the *n_qubits* as needed.

1.2 Critic structure

The output of the quantum circuit is a state vector $|\psi\rangle$. To get an image, you have two main options:

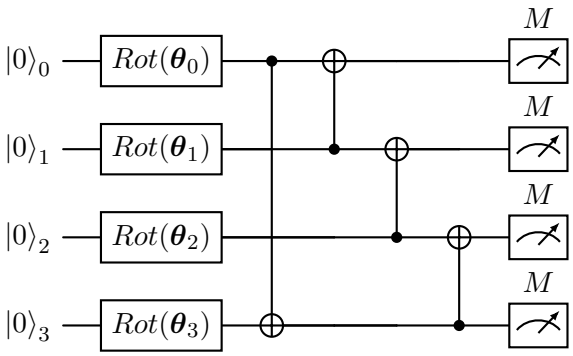


Fig. 3.2: Strongly Entangling Ansatz circuit for 4 qubits. $\boldsymbol{\theta}_i$ represents the three Euler angles for wire i (`params[0][i]`), and M represents the observable.

1. *Probability Distribution*: Use the square of the amplitudes of the state vector as your pixel values.

$$P(i) = |\langle i|\psi\rangle|^2 \tag{3.1}$$

2 Training Strategy

2.1 Training objective

2.2 Training algorithm

3 Experimental Setup

3.1 Dataset and libraries

Experimentations are conducted on the MNIST and Fashion-MNIST datasets, which consist of 28x28 grayscale images of handwritten digits and fashion items, respectively. The implementation of the quantum circuits and training algorithms is carried out on Python3, using the PennyLane library[15], which provides a high-level interface for quantum machine learning and supports various quantum hardware backends, and the PyTorch library[16] for classical optimization and data handling.

3.2 Hyperparameters

Results

1 Results Analysis

2 Discussion

Conclusion

Bibliography

[1] P.-L. Dallaire-Demers and N. Killoran, "Quantum generative adversarial networks," *Physical Review A*, vol. 98, July 2018.

[2] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial networks," 2014.

[3] L. Mescheder, A. Geiger, and S. Nowozin, "Which training methods for gans do actually converge?," 2018.

[4] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein gan," 2017.

[5] T. Karras, T. Aila, S. Laine, and J. Lehtinen, "Progressive growing of gans for improved quality, stability, and variation," 2018.

[6] S. Lloyd and C. Weedbrook, "Quantum generative adversarial learning," *Physical Review Letters*, vol. 121, July 2018.

[7] M. Benedetti, E. Lloyd, S. Sack, and M. Fiorentini, "Parameterized quantum circuits as machine learning models," *Quantum Science and Technology*, vol. 4, p. 043001, Nov. 2019.

[8] S. L. Tsang, M. T. West, S. M. Erfani, and M. Usman, "Hybrid quantumclassical generative adversarial network for high-resolution image generation," *IEEE Transactions on Quantum Engineering*, vol. 4, p. 119, 2023.

[9] J. Jäger, F. J. Kiwit, and C. A. Riofrío, "Scaling quantum machine learning without tricks: High-resolution and diverse image generation," 2026.

[10] H.-L. Huang, Y. Du, M. Gong, Y. Zhao, Y. Wu, C. Wang, S. Li, F. Liang, J. Lin, Y. Xu, R. Yang, T. Liu, M.-H. Hsieh, H. Deng, H. Rong, C.-Z. Peng, C.-Y. Lu, Y.-A. Chen, D. Tao, X. Zhu, and J.-W. Pan, "Experimental quantum generative adversarial networks for image generation," *Physical Review Applied*, vol. 16, Aug. 2021.

[11] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2022.

[12] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep unsupervised learning using nonequilibrium thermodynamics," 2015.

[13] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, "Improved training of wasserstein gans," 2017.

[14] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles, "Variational quantum algorithms," *Nature Reviews Physics*, vol. 3, p. 625644, Aug. 2021.

[15] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, S. Ahmed, V. Ajith, M. S. Alam, G. Alonso-Linaje, B. AkashNarayanan, A. Asadi, J. M. Arrazola, U. Azad, S. Banning, C. Blank, T. R. Bromley, B. A. Cordier, J. Ceroni, A. Delgado, O. D. Matteo, A. Dusko, T. Garg, D. Guala, A. Hayes, R. Hill, A. Ijaz, T. Isacsson, D. Ittah, S. Jahangiri, P. Jain, E. Jiang, A. Khandelwal, K. Kottmann, R. A. Lang, C. Lee, T. Loke, A. Lowe, K. McKiernan, J. J. Meyer, J. A. Montañez-Barrera, R. Moyard, Z. Niu, L. J. O'Riordan, S. Oud, A. Panigrahi, C.-Y. Park, D. Polatajko, N. Quesada, C. Roberts, N. Sá, I. Schoch, B. Shi, S. Shu, S. Sim, A. Singh, I. Strandberg, J. Soni, A. Száva, S. Thabet, R. A. Vargas-Hernández, T. Vincent, N. Vitucci, M. Weber, D. Wierichs, R. Wiersema, M. Willmann, V. Wong, S. Zhang, and N. Killoran, "Pennylane: Automatic differentiation of hybrid quantum-classical computations," 2022.

[16] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," 2019.

Appendix

1 Exemple

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum

wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Annexe

1 Cost functions

2 Measurement

3 Optimizers

4 Gradient descent